

■ ROTEIRO — VÍDEO PITCH

Sistema Automotivo · Gestão de Estoque de Veículos

■■ FALA	O que você diz para a câmera
■■ TELA	O que deve aparecer na gravação de tela
■ DICA	Orientação de apresentação

Duração Total	≈ 4 minutos
Parte 1 — Introdução	0:00 → 1:00 (60 segundos)
Parte 2 — Código e Arquitetura	1:00 → 2:00 (60 segundos)
Parte 3 — Demonstração ao Vivo	2:00 → 4:00 (120 segundos)
Ferramenta de teste	Swagger UI (http://localhost:8080/swagger-ui.html)

PARTE 1 — INTRODUÇÃO AO PROBLEMA E AO SISTEMA

■ ■ 00:00 → 01:00 (60 segundos)

■ ■ "Olá! Meu nome é [SEU NOME] e hoje vou apresentar o Sistema Automotivo — uma aplicação de Gestão de Estoque de Veículos, desenvolvida como projeto da disciplina de Programação Orientada a Objetos da UniFECAF."

■ ■ "O problema que motivou este projeto é bem real: muitas concessionárias ainda gerenciam seu estoque de forma manual — em planilhas ou cadernos — o que gera desorganização, informações desatualizadas e perda de vendas."

■ ■ "Pensando nisso, desenvolvemos uma API REST completa que permite: cadastrar veículos com todas as suas informações, consultar e filtrar o estoque por marca, modelo, preço, ano e status, além de atualizar e remover registros com facilidade."

■ ■ "A solução foi construída com Java 17, Spring Boot 3 e banco de dados MySQL, seguindo o padrão arquitetural MVC com API REST — tecnologias amplamente utilizadas no mercado de desenvolvimento brasileiro."

■ ■ *TELA: Mostre o diagrama de entidades (Marca → Modelo → Veiculo) ou o README do projeto aberto no VS Code enquanto fala.*

■ ■ *DICA: Fale com confiança e clareza. Não leia este roteiro — use-o como guia. Olhe para a câmera ao descrever o problema.*

PARTE 2 — EXPLICAÇÃO DO CÓDIGO E ARQUITETURA

■ ■ 01:00 → 02:00 (60 segundos)

■ ■ "Agora vou mostrar como o projeto está estruturado. Seguimos a arquitetura em três camadas, também chamada de MVC adaptado para APIs REST."

■ ■ "Na camada de Model, temos nossas entidades JPA: Marca, Modelo e Veiculo. Destaco aqui a classe abstrata VeiculoBase — ela demonstra na prática o conceito de Herança da POO. A classe Veiculo a estende e implementa o método getTipoVeiculo(), o que configura Polimorfismo."

■ ■ *TELA: Abra o arquivo VeiculoBase.java no VS Code. Destaque com o cursor a assinatura do método abstrato getTipoVeiculo(). Em seguida, abra Veiculo.java e mostre o @Override do mesmo método.*

■ ■ "Na camada de Repository, usamos o Spring Data JPA. Veja que a interface VeiculoRepository apenas declara os métodos — e o Spring gera toda a implementação automaticamente. Isso é a Abstração em ação."

■ ■ *TELA: Mostre VeiculoRepository.java. Destaque o método findWithFilters() com a anotação @Query e os parâmetros opcionais.*

■ ■ "Por fim, na camada de Service, ficam todas as regras de negócio — validação de ano de fabricação, controle de preço, proteção de exclusão. E no Controller, os endpoints REST recebem as requisições HTTP e delegam ao Service. Tudo desacoplado."

■■ TELA: Mostre rapidamente o `VeiculoController.java`, destacando os métodos `@GetMapping("/filtrar")` e `@PostMapping`.

■ DICA: Mantenha o editor de código em fonte grande (`Ctrl+=` no VS Code) para o código ficar legível na gravação.

PARTE 3 — DEMONSTRAÇÃO AO VIVO (CRUD + FILTROS)

■■ 02:00 → 04:00 (120 segundos)

■■ "Agora vamos ver o sistema funcionando. Estou usando o Swagger UI, que é gerado automaticamente pelo Springdoc. Basta acessar `localhost:8080/swagger-ui.html`."

■■ TELA: Abra o browser em `http://localhost:8080/swagger-ui.html`. Mostre os três grupos de endpoints: Marcas, Modelos e Veículos.

■■ 02:15 → CREATE (POST)

■■ "Primeiro, vou cadastrar um novo veículo. Abro o endpoint `POST /api/veiculos/modelo/{modeloid}`, clico em Try it out, coloco `modeloid = 1` (Toyota Corolla) e envio este JSON:"

■■ TELA: Cole este JSON no campo request body do Swagger: `{ "anoFabricacao": 2024, "cor": "Azul Metálico", "preco": 135000.00, "quilometragem": 0, "status": "DISPONIVEL", "observacoes": "Veículo 0km, pronto para entrega" }` Execute e mostre o retorno HTTP 201 com o ID gerado.

■ Anote o ID retornado — você vai usá-lo nos próximos passos.

■■ 02:40 → READ (GET — listar todos)

■■ "Agora vejo todo o estoque. Chamo o `GET /api/veiculos` e recebo a lista completa "

■■ TELA: Execute o `GET /api/veiculos`. Mostre a lista retornada, incluindo o veículo que acabou de cadastrar.

■■ 02:55 → READ FILTRADO (GET /filtrar) — O ponto alto da demo!

■■ "Aqui está um dos diferenciais do sistema: o filtro combinado. Posso buscar, por exemplo, todos os veículos DISPONÍVEIS da Toyota com preço até 150 mil reais. Veja o resultado em tempo real:"

■■ TELA: No Swagger, abra o `GET /api/veiculos/filtrar`. Preencha: `marca = Toyota`, `status = DISPONIVEL`, `precoMax = 150000`. Execute. Mostre os resultados filtrados. Em seguida, faça uma segunda busca: `ano = 2024`, `status = DISPONIVEL`. Mostre que só aparecem os do ano correto.

■ Este é o momento mais importante da demo. Pause um segundo após mostrar o resultado filtrado.

■■ 03:20 → UPDATE (PUT)

■■ "Agora vou simular a atualização de status de um veículo que foi vendido."

■■ TELA: Abra o `PUT /api/veiculos/{id}`. Use o ID do veículo criado. Envie: `{ "status": "VENDIDO", "preco": 130000.00 }` Mostre o retorno HTTP 200 com o veículo atualizado.

■■ 03:40 → DELETE

■■ "Por último, vou remover um veículo cadastrado incorretamente."

■■ TELA: Abra o `DELETE /api/veiculos/{id}`. Use o ID do veículo criado. Execute. Mostre o HTTP 204 (No Content) — confirmando a exclusão. Em seguida, tente um `GET /api/veiculos/{id}` com o mesmo

ID e mostre o HTTP 404.

■ O 404 depois do delete prova que a exclusão funcionou corretamente.

■ ■ 03:55 → **ENCERRAMENTO**

■ ■ "E assim concluímos a demonstração do Sistema Automotivo. O projeto aplica na prática os conceitos de POO — herança, encapsulamento, abstração e polimorfismo — dentro de uma arquitetura REST profissional com Spring Boot e MySQL. O código-fonte completo está disponível no GitHub no link da descrição. Obrigado!"

■ ■ **TELA:** Volte para o Swagger UI aberto mostrando os três grupos de endpoints.

■ **DICA FINAL:** Grave o áudio com fone de ouvido para melhor qualidade. Use OBS Studio, Loom ou a gravação de tela do Windows (Win+G) para capturar a tela. Fale devagar e claramente. Não precisa ser perfeito — o importante é mostrar que o sistema funciona!